Zeppelin Notebook Job Search anonymous

Avance del Proyecto Bim...

Head 1

default

📊 Análisis Exploratorio de Datos (EDA)

Objetivo:
Realizar un Análisis Exploratorio de Datos (EDA) completo al conjunto de datos que se utilizará en el proyecto bimestral.

🔍Integrantes:
-Juan Pablo Landi
-María Valentina Samaniego
-María Paula Guallo

Took 3 seconds. Last updated by anonymous at July 19 2025, 8:52:03 PM.

FINISHED

1 Lectura del archivo

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:52:03 PM.

SPARK JOB FINISHED

```
val dfNacidos = spark
    .read
    .option("header", true)
    .option("sep", ";")
    .option("inferSchema" , true)
    .csv("/workspace/pavan-s10/ENV_2023 - copia.csv")
```

```
dfNacidos: org.apache.spark.sql.DataFrame = [prov_insc: string, cant_insc: string ... 45 more fields]
```

Took 21 seconds. Last updated by anonymous at July 19 2025, 8:52:25 PM.

FINISHED

2 Imprimir el Esquema del Dataset

-Propósito:
Verificar si la interfaz de lectura ha procesado correctamente los tipos de datos de cada columna.

Referencia:
Según el archivo `inec_nacidosvivos_dd_2023.ods`, algunas columnas esperadas como *numéricas* son:

- anio_nac (Año de nacimiento)
- dia_nac (Día de nacimiento)
- anio_insc (¿Año de inscripción?)
- mes_insc (¿Mes de inscripción?)
- dia_insc (¿Día de inscripción?)
- talla (¿Talla del recién nacido?)
- peso (¿Peso del recién nacido?)

📌 Esta validación es clave para evitar errores posteriores durante el análisis estadístico o modelado.

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:52:25 PM.

FINISHED

```
dfNacidos.printSchema
```

```
root
 |-- prov_insc: string (nullable = true)
 |-- cant_insc: string (nullable = true)
 |-- parr_insc: string (nullable = true)
 |-- fecha_insc: string (nullable = true)
 |-- anio_insc: string (nullable = true)
 |-- mes_insc: string (nullable = true)
 |-- dia_insc: string (nullable = true)
 |-- sexo: string (nullable = true)
 |-- fecha_nac: string (nullable = true)
```

```
|-- anio_nac: integer (nullable = true)
|-- mes_nac: string (nullable = true)
|-- dia_nac: integer (nullable = true)
|-- talla: string (nullable = true)
|-- peso: string (nullable = true)
|-- sem_gest: string (nullable = true)
|-- tipo_part: string (nullable = true)
|-- lugar_ocur: string (nullable = true)
|-- apgar1: string (nullable = true)
|-- apgar5: string (nullable = true)
|-- p_emb: string (nullable = true)
|-- prov_nac: string (nullable = true)
|-- cant_nac: string (nullable = true)
|-- parr_nac: string (nullable = true)
|-- area_nac: string (nullable = true)
|-- asis_por: string (nullable = true)
|-- nac_mad: string (nullable = true)
|-- cod_pais: string (nullable = true)
|-- fecha_mad: string (nullable = true)
|-- anio_mad: string (nullable = true)
|-- mes_mad: string (nullable = true)
|-- dia_mad: string (nullable = true)
|-- edad_mad: string (nullable = true)
|-- con_pren: string (nullable = true)
|-- num_emb: string (nullable = true)
|-- num_par: string (nullable = true)
|-- hij_viv: integer (nullable = true)
|-- hij_vivm: string (nullable = true)
|-- hij_nacm: string (nullable = true)
|-- etnia: string (nullable = true)
|-- est_civil: string (nullable = true)
|-- niv_inst: string (nullable = true)
|-- sabe_leer: string (nullable = true)
|-- prov_res: string (nullable = true)
|-- cant_res: string (nullable = true)
|-- parr_res: string (nullable = true)
|-- area_res: string (nullable = true)
|-- residente: string (nullable = true)
```

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:52:25 PM.

FINISHED

3 Imprimir Cantidad de Filas y Columnas

🔢 *Propósito:*
Verificar que los datos se hayan cargado correctamente, comparando las dimensiones del dataset con lo especificado en la documentación oficial de origen.

✅ *Acción:*
Imprimir el número total de *filas* (registros) y *columnas* (variables).

📌 Esta validación inicial es esencial para confirmar que no hubo errores al momento de la lectura del archivo o la conexión con la fuente de datos.

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:52:25 PM.

SPARK JOB FINISHED

```
print(s"Registros (filas): ${dfNacidos.count}, Variables (columnas): ${dfNacidos.columns.length}")
```

```
Registros (filas): 241295, Variables (columnas): 47
```

Took 1 second. Last updated by anonymous at July 19 2025, 8:52:27 PM.

FINISHED

4 Estadística Descriptiva

-Objetivo:
Resumir y describir las principales características de los datos mediante medidas estadísticas básicas.

-Enfoque 1: describe()
Utilizaremos el método describe() para obtener un resumen estadístico de las columnas numéricas (y, si es posible, también de algunas categóricas).

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:52:27 PM.

SPARK JOB FINISHED

```
dfNacidos
    .describe()
    .show
```

```
+-------+----------------+---------+--------------------+----------+------------------+----------+------------------+------+----------+----------------
|summary|       prov_insc|cant_insc|           parr_insc|fecha_insc|         anio_insc|  mes_insc|          dia_insc|  sexo| fecha_nac|          anio_n
+-------+----------------+---------+--------------------+----------+------------------+----------+------------------+------+----------+----------------
|  count|          241295|   241295|              241295|    241295|            241295|    241295|            241295|241295|    241295|            2412
|   mean|            null|     null|                null|      null|2023.0647378454437|      null|15.856224576692474|  null|      null|2022.93723450548
| stddev|            null|     null|                null|      null|0.6836599754432079|      null| 8.685787511632697|  null|      null|1.51805304711637
|    min|                |         |                    |          |                  |          |                  |Hombre|1900/01/01|              19
|    max|Zamora Chinchipe|   Zaruma|Ángel Polibio Cháves|2024/04/30|              2024|Septiembre|                 9| Mujer|2023/12/31|              20
+-------+----------------+---------+--------------------+----------+------------------+----------+------------------+------+----------+----------------
```

Took 29 seconds. Last updated by anonymous at July 19 2025, 8:52:56 PM.

SPARK JOB FINISHED

```
dfNacidos
    .select("anio_insc","dia_insc","anio_nac","dia_nac")
    .describe()
    .show
```

```
+-------+------------------+------------------+------------------+------------------+
|summary|         anio_insc|          dia_insc|          anio_nac|          dia_nac|
+-------+------------------+------------------+------------------+------------------+
|  count|            241295|            241295|            241295|            241295|
|   mean|2023.0647378454437|15.856224576692474|2022.9372345054808|15.632362875318593|
| stddev|0.6836599754432079| 8.685787511632697|1.5180530471163742| 8.784520746500172|
|    min|                 |                 |              1900|                 1|
|    max|              2024|                 9|              2023|                31|
+-------+------------------+------------------+------------------+------------------+
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:52:58 PM.

FINISHED

💡 Alternativa cuando hay muchas columnas

La anterior es un alatenativa, pero, si la cantidad de columnas es muy gande¿que se puede hacer?.
Una idea es coger unicamnete las columnas que tiene un tipo de concreto, por ejemplo numerico.

Took 1 second. Last updated by anonymous at July 19 2025, 8:52:59 PM.

SPARK JOB FINISHED

```
import org.apache.spark.sql.types._

val numericCols = dfNacidos.schema.fields.filter{
    case StructField(_, IntegerType | LongType | FloatType | DoubleType | ShortType | DecimalType(), _, _) => true
    case _ => false
}.map(_.name)

dfNacidos.select(numericCols.map(col):_*).describe().show
```

```
+-------+------------------+------------------+------------------+
|summary|          anio_nac|           dia_nac|           hij_viv|
+-------+------------------+------------------+------------------+
|  count|            241295|            241295|            241295|
|   mean|2022.9372345054808|15.632362875318593|2.0577550301498166|
| stddev|1.5180530471163742| 8.784520746500172|1.2242374055754184|
|    min|              1900|                 1|                 1|
|    max|              2023|                31|                16|
+-------+------------------+------------------+------------------+

import org.apache.spark.sql.types._
numericCols: Array[String] = Array(anio_nac, dia_nac, hij_viv)
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:53:01 PM.

SPARK JOB FINISHED

```
dfNacidos.select(numericCols.map(col):_*).summary().show
```

```
+-------+------------------+------------------+------------------+
|summary|          anio_nac|           dia_nac|           hij_viv|
```

```
+-------+------------------+------------------+------------------+
|  count|            241295|            241295|            241295|
|   mean|2022.9372345054808|15.632362875318593|2.0577550301498166|
| stddev|1.5180530471163742| 8.784520746500172|1.2242374055754184|
|    min|              1900|                 1|                 1|
|    25%|              2023|                 8|                 1|
|    50%|              2023|                16|                 2|
|    75%|              2023|                23|                 3|
|    max|              2023|                31|                16|
+-------+------------------+------------------+------------------+
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:53:03 PM.

SPARK JOB FINISHED

```
dfNacidos.select(numericCols.map(col):_*).summary("stddev","25%").show
```

```
+-------+------------------+-----------------+------------------+
|summary|          anio_nac|          dia_nac|           hij_viv|
+-------+------------------+-----------------+------------------+
| stddev|1.5180530471163742|8.784520746500172|1.2242374055754184|
|    25%|              2023|                8|                 1|
+-------+------------------+-----------------+------------------+
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:53:06 PM.

SPARK JOB FINISHED

```
dfNacidos.select(numericCols.map(col):_*).summary("stddev","91%").show
```

```
+-------+------------------+-----------------+------------------+
|summary|          anio_nac|          dia_nac|           hij_viv|
+-------+------------------+-----------------+------------------+
| stddev|1.5180530471163742|8.784520746500172|1.2242374055754184|
|    91%|              2023|               28|                 4|
+-------+------------------+-----------------+------------------+
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:53:08 PM.

SPARK JOB FINISHED

```
dfNacidos.select(numericCols.map(col):_*).summary("count","count_distinct").show
```

```
+--------------+--------+-------+-------+
|       summary|anio_nac|dia_nac|hij_viv|
+--------------+--------+-------+-------+
|         count|  241295| 241295| 241295|
|count_distinct|      41|     31|     15|
+--------------+--------+-------+-------+
```

Took 2 seconds. Last updated by anonymous at July 19 2025, 8:53:10 PM.

FINISHED

TRANSFORMACIONES

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:53:10 PM.

FINISHED

```
val dfNacidosClean = dfNacidos.withColumn("fecha_insc_date",to_date(col("fecha_insc"),"yyyy/MM/dd"))
```

```
dfNacidosClean: org.apache.spark.sql.DataFrame = [prov_insc: string, cant_insc: string ... 46 more fields]
```

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:53:10 PM.

FINISHED

```
dfNacidosClean.printSchema
```

```
root
 |-- prov_insc: string (nullable = true)
 |-- cant_insc: string (nullable = true)
 |-- parr_insc: string (nullable = true)
 |-- fecha_insc: string (nullable = true)
```

```
 |-- anio_insc: string (nullable = true)
 |-- mes_insc: string (nullable = true)
 |-- dia_insc: string (nullable = true)
 |-- sexo: string (nullable = true)
 |-- fecha_nac: string (nullable = true)
 |-- anio_nac: integer (nullable = true)
 |-- mes_nac: string (nullable = true)
 |-- dia_nac: integer (nullable = true)
 |-- talla: string (nullable = true)
 |-- peso: string (nullable = true)
 |-- sem_gest: string (nullable = true)
 |-- tipo_part: string (nullable = true)
 |-- lugar_ocur: string (nullable = true)
 |-- apgar1: string (nullable = true)
 |-- apgar5: string (nullable = true)
 |-- p_emb: string (nullable = true)
 |-- prov_nac: string (nullable = true)
 |-- cant_nac: string (nullable = true)
 |-- parr_nac: string (nullable = true)
 |-- area_nac: string (nullable = true)
 |-- asis_por: string (nullable = true)
 |-- nac_mad: string (nullable = true)
 |-- cod_pais: string (nullable = true)
 |-- fecha_mad: string (nullable = true)
 |-- anio_mad: string (nullable = true)
 |-- mes_mad: string (nullable = true)
 |-- dia_mad: string (nullable = true)
 |-- edad_mad: string (nullable = true)
 |-- con_pren: string (nullable = true)
 |-- num_emb: string (nullable = true)
 |-- num_par: string (nullable = true)
 |-- hij_viv: integer (nullable = true)
 |-- hij_vivm: string (nullable = true)
 |-- hij_nacm: string (nullable = true)
 |-- etnia: string (nullable = true)
 |-- est_civil: string (nullable = true)
 |-- niv_inst: string (nullable = true)
 |-- sabe_leer: string (nullable = true)
 |-- prov_res: string (nullable = true)
 |-- cant_res: string (nullable = true)
 |-- parr_res: string (nullable = true)
 |-- area_res: string (nullable = true)
 |-- residente: string (nullable = true)
 |-- fecha_insc_date: date (nullable = true)
```

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:53:10 PM.

FINISHED

```
val dfNacidosClean = dfNacidos
    .withColumn("fecha_insc_date",to_date(col("fecha_insc"),"yyyy/MM/dd"))
    .withColumn("fecha_nac_date",to_date(col("fecha_nac"),"yyyy/MM/dd"))
    .withColumn("fecha_mad_date",to_date(col("fecha_mad"),"yyyy/MM/dd"))
```

```
dfNacidosClean: org.apache.spark.sql.DataFrame = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 19 2025, 8:53:10 PM.

PENDING

```
val colsProNumber = Seq("anio_insc", "dia_insc", "talla", "peso", "sem_gest", "apgar1", "apgar5", "anio_mad", "dia_mad", "edad_mad", "con_pren", "num
```

FINISHED

```
val dfNacidosCleanFinal = colsProNumber
    .foldLeft(dfNacidosClean) { (dfTemp, colName) =>
    dfTemp.withColumn(colName, col(colName).cast("int"))
  }
```

```
dfNacidosCleanFinal: org.apache.spark.sql.DataFrame = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:30:15 AM.

FINISHED

```
dfNacidosClean.printSchema
```

```
root
 |-- prov_insc: string (nullable = true)
```

```
|-- cant_insc: string (nullable = true)
|-- parr_insc: string (nullable = true)
|-- fecha_insc: string (nullable = true)
|-- anio_insc: string (nullable = true)
|-- mes_insc: string (nullable = true)
|-- dia_insc: string (nullable = true)
|-- sexo: string (nullable = true)
|-- fecha_nac: string (nullable = true)
|-- anio_nac: integer (nullable = true)
|-- mes_nac: string (nullable = true)
|-- dia_nac: integer (nullable = true)
|-- talla: string (nullable = true)
|-- peso: string (nullable = true)
|-- sem_gest: string (nullable = true)
|-- tipo_part: string (nullable = true)
|-- lugar_ocur: string (nullable = true)
|-- apgar1: string (nullable = true)
|-- apgar5: string (nullable = true)
|-- p_emb: string (nullable = true)
|-- prov_nac: string (nullable = true)
|-- cant_nac: string (nullable = true)
|-- parr_nac: string (nullable = true)
|-- area_nac: string (nullable = true)
|-- asis_por: string (nullable = true)
|-- nac_mad: string (nullable = true)
|-- cod_pais: string (nullable = true)
|-- fecha_mad: string (nullable = true)
|-- anio_mad: string (nullable = true)
|-- mes_mad: string (nullable = true)
|-- dia_mad: string (nullable = true)
|-- edad_mad: string (nullable = true)
|-- con_pren: string (nullable = true)
|-- num_emb: string (nullable = true)
|-- num_par: string (nullable = true)
|-- hij_viv: integer (nullable = true)
|-- hij_vivm: string (nullable = true)
|-- hij_nacm: string (nullable = true)
|-- etnia: string (nullable = true)
|-- est_civil: string (nullable = true)
|-- niv_inst: string (nullable = true)
|-- sabe_leer: string (nullable = true)
|-- prov_res: string (nullable = true)
|-- cant_res: string (nullable = true)
|-- parr_res: string (nullable = true)
|-- area_res: string (nullable = true)
|-- residente: string (nullable = true)
|-- fecha_insc_date: date (nullable = true)
|-- fecha_nac_date: date (nullable = true)
|-- fecha_mad_date: date (nullable = true)
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 11:06:21 AM.

SPARK JOB FINISHED

```
import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._

val dateColumnNames = dfNacidosClean
    .schema
    .fields
    .filter {
        case StructField(_,TimestampType | DateType,_,_) => true
        case _ => false
    }
    .map(_.name)

// Ejemplo de calculo de estadisticas comunes
val statsExprs = dateColumnNames
    .flatMap { colName =>
        Seq(
            count(col(colName)).alias(s"${colName}_count"),
            min(col(colName)).alias(s"${colName}_min"),
            max(col(colName)).alias(s"${colName}_max"),
            countDistinct(col(colName)).alias(s"${colName}_countDistinct")
        )
    }

val dfStatsDateCols = dfNacidosClean
    .select(statsExprs:_*)

dfStatsDateCols.show()
```

```
+--------------------+-------------------+-------------------+----------------------------+--------------------+-------------------+-------------------
|fecha_insc_date_count|fecha_insc_date_min|fecha_insc_date_max|fecha_insc_date_countDistinct|fecha_nac_date_count|fecha_nac_date_min|fecha_nac_date_max
+--------------------+-------------------+-------------------+----------------------------+--------------------+-------------------+-------------------
|              231333|         2001-01-01|         2024-04-30|                         452|              241295|         1900-01-01|         2023-12-31
```

```
+--------------------+------------------+------------------+--------------------------+------------------+-----------------+-----------------
```

```
import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._
dateColumnNames: Array[String] = Array(fecha_insc_date, fecha_nac_date, fecha_mad_date)
statsExprs: Array[org.apache.spark.sql.Column] = Array(count(fecha_insc_date) AS fecha_insc_date_count, min(fecha_insc_date) AS fecha_insc_date_min, ma
```

Took 5 seconds. Last updated by anonymous at June 25 2025, 6:55:16 PM.

FINISHED

```
import org.apache.spark.sql.types._

val columnasEnteras = dfNacidosClean.schema.fields
    .filter(_.dataType == IntegerType)
    .map(_.name)

columnasEnteras.foreach(println)
```

```
anio_nac
dia_nac
hij_viv
import org.apache.spark.sql.types._
columnasEnteras: Array[String] = Array(anio_nac, dia_nac, hij_viv)
```

Took 1 second. Last updated by anonymous at July 16 2025, 7:30:57 PM.

FINISHED

```
val columnasString = dfNacidosClean.schema.fields
    .filter(_.dataType == StringType)
    .map(_.name)
columnasString.foreach(println)
```

```
prov_insc
cant_insc
parr_insc
fecha_insc
anio_insc
mes_insc
dia_insc
sexo
fecha_nac
mes_nac
talla
peso
sem_gest
tipo_part
lugar_ocur
apgar1
apgar5
p_emb
prov_nac
cant_nac
parr_nac
area_nac
asis_por
nac_mad
cod_pais
fecha_mad
anio_mad
mes_mad
dia_mad
edad_mad
con_pren
num_emb
num_par
hij_vivm
hij_nacm
etnia
est_civil
niv_inst
sabe_leer
prov_res
cant_res
parr_res
area_res
residente
columnasString: Array[String] = Array(prov_insc, cant_insc, parr_insc, fecha_insc, anio_insc, mes_insc, dia_insc, sexo, fecha_nac, mes_nac, talla, peso
```

Took 0 seconds. Last updated by anonymous at June 25 2025, 7:21:41 PM.

SPARK JOB FINISHED

```
dfNacidosClean.groupBy("anio_nac","dia_nac","hij_viv")
    .count()
    .orderBy(desc("count"))
    .show
```

```
+--------+-------+-------+-----+
|anio_nac|dia_nac|hij_viv|count|
+--------+-------+-------+-----+
|    2023|      6|      1| 3376|
|    2023|      7|      1| 3316|
|    2023|     14|      1| 3302|
|    2023|     18|      1| 3295|
|    2023|     13|      1| 3282|
|    2023|     28|      1| 3279|
|    2023|     27|      1| 3274|
|    2023|     20|      1| 3265|
|    2023|     10|      1| 3255|
|    2023|     23|      1| 3224|
|    2023|      8|      1| 3224|
|    2023|      4|      1| 3222|
|    2023|     11|      1| 3220|
|    2023|     17|      1| 3208|
|    2023|     16|      1| 3196|
|    2023|     15|      1| 3192|
|    2023|     19|      1| 3178|
|    2023|      3|      1| 3177|
|    2023|     26|      1| 3175|
|    2023|      1|      1| 3171|
+--------+-------+-------+-----+
only showing top 20 rows
```

Took 1 second. Last updated by anonymous at June 25 2025, 7:44:48 PM.

FINISHED

TALLER GRUPAL 1

El objetivo de este trabajo fue el poder agrupar las columnas por tipos de datos para asi poder procesar los datos de mejor manera en un futuro, de esta manera adquirimos mas conocimientos para poder realizar el proyecto de mejor manera

Took 0 seconds. Last updated by anonymous at June 25 2025, 8:40:27 PM.

SPARK JOB FINISHED

```
import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._

val columnasStringNumericas = Seq("Mes_nac", "Talla", "Peso", "Hij_Vivm", "Hij_Nacm")

val statsExprsStringNum = columnasStringNumericas.flatMap { colName =>
  Seq(
    count(col(colName)).alias(s"${colName}_count"),
    countDistinct(col(colName)).alias(s"${colName}_countDistinct")
  )
}

val dfStatsStringNumericCols = dfNacidosClean.select(statsExprsStringNum: _*)
dfStatsStringNumericCols.show()
```

```
+-------------+---------------------+-----------+-------------------+----------+------------------+-------------+----------------------+-------------
|Mes_nac_count|Mes_nac_countDistinct|Talla_count|Talla_countDistinct|Peso_count|Peso_countDistinct|Hij_Vivm_count|Hij_Vivm_countDistinct|Hij_Nacm_count
+-------------+---------------------+-----------+-------------------+----------+------------------+-------------+----------------------+-------------
|       241295|                   12|     241295|                 19|    241295|              2969|       241295|                    11|       241295
+-------------+---------------------+-----------+-------------------+----------+------------------+-------------+----------------------+-------------

import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._
columnasStringNumericas: Seq[String] = List(Mes_nac, Talla, Peso, Hij_Vivm, Hij_Nacm)
statsExprsStringNum: Seq[org.apache.spark.sql.Column] = List(count(Mes_nac) AS Mes_nac_count, count(Mes_nac) AS Mes_nac_countDistinct, count(Talla) AS
dfStatsStringNumericCols: org.apache.spark.sql.DataFrame = [Mes_nac_count: bigint, Mes_nac_countDistinct: bigint ... 8 more fields]
```

Took 1 second. Last updated by anonymous at July 16 2025, 8:52:02 PM. (outdated)

FINISHED

TALLER GRUPAL 2

El objetivo de este taller ahora será trabajar con los tipos de datos que son cadenas o que también se las conoce como 'String'
Aqui realizaremos el EDA de las columnas que poseen estos tipos de datos

Took 2 seconds. Last updated by anonymous at July 02 2025, 6:13:30 PM.

FINISHED

```
val columnasCategoricas = Seq(
  "prov_insc", "cant_insc", "parr_insc", "mes_insc", "sexo",
  "mes_nac", "tipo_part", "p_emb", "lugar_ocur", "prov_nac",
  "cant_nac", "parr_nac", "area_nac", "asis_por", "nac_mad",
  "cod_pais", "etnia", "est_civil", "niv_inst", "sabe_leer",
  "prov_res", "cant_res", "par_res", "area_res", "residente"
)
```

```
columnasCategoricas: Seq[String] = List(prov_insc, cant_insc, parr_insc, mes_insc, sexo, mes_nac, tipo_part, p_emb, lugar_ocur, prov_nac, cant_nac, par
```

Took 0 seconds. Last updated by anonymous at July 02 2025, 6:37:48 PM.

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("cod_pais")
    .groupBy("cod_pais")
    .count()
}
```

Setting

cod_pais	count
Paraguay	8
Canadá	4
Suiza	5
Italia	7
Egipto, Rep. ?rabe de	1
Francia	13
República Checa	1
España	37
Uzbekistán	2
Haití	8

Export

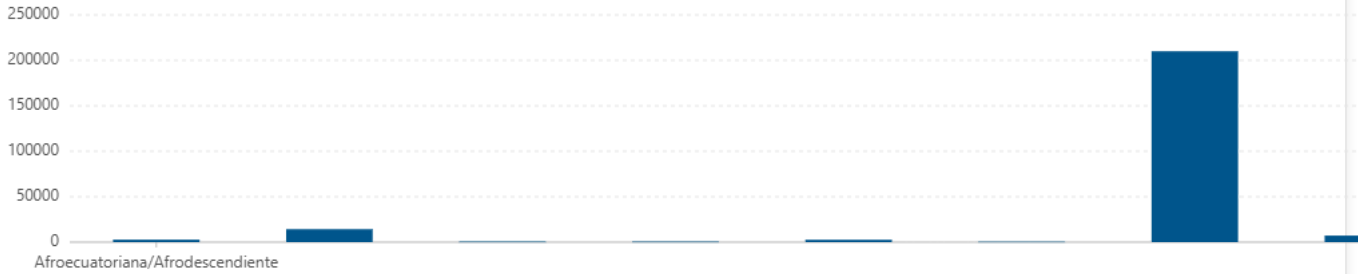
1 2 3 4 5 6 7 10 / page

Took 1 second. Last updated by anonymous at July 02 2025, 6:44:24 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("etnia")
    .groupBy("etnia")
    .count()
}
```

Setting



Took 2 seconds. Last updated by anonymous at July 02 2025, 6:44:58 PM.

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("etnia")
    .distinct
    .groupBy("etnia")
    .count()
}
```

Setting

etnia	count
Afroecuatoriana/Afrodescendiente	1
Indígena	1
Blanca	1
Otra	1
Negra	1
Mulata	1
Mestiza	1
Sin información	1
Montubia	1

Export

1

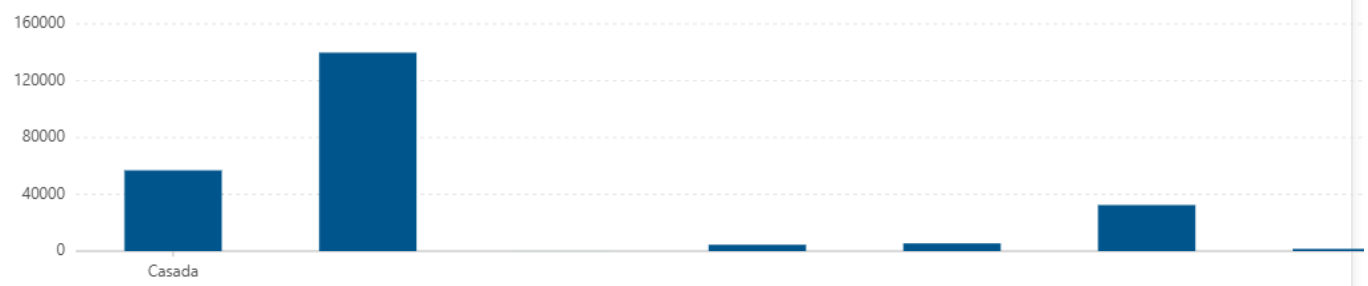
10 / page

Took 2 seconds. Last updated by anonymous at July 02 2025, 7:37:57 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("est_civil")
    .groupBy("est_civil")
    .count()
}
```

Setting

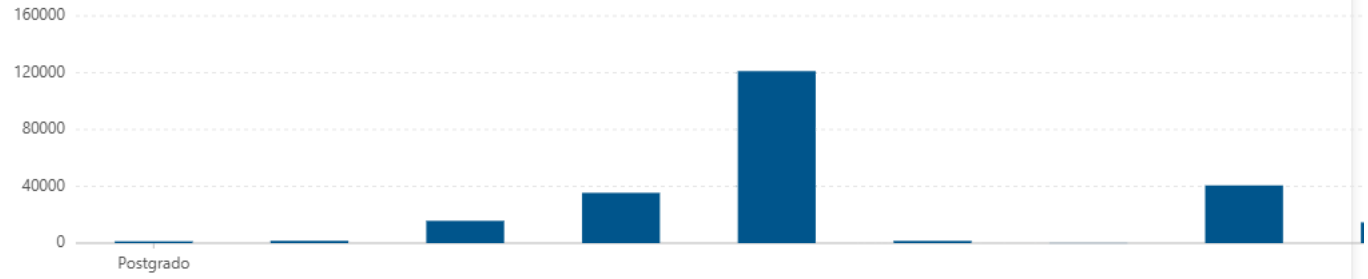


Took 1 second. Last updated by anonymous at July 02 2025, 6:45:28 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("niv_inst")
    .groupBy("niv_inst")
    .count()
}
```

Setting



Took 1 second. Last updated by anonymous at July 02 2025, 6:46:06 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("sabe_leer")
    .groupBy("sabe_leer")
    .count()
}
```

Setting

Took 1 second. Last updated by anonymous at July 02 2025, 6:46:36 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
```

```
  dfNacidosClean
    .select("prov_res")
    .groupBy("prov_res")
    .count()
}
```

Setting

prov_res	count
Sucumbíos	3721
Cotopaxi	6499
Bolívar	2529
Pichincha	34259
Chimborazo	6038
Morona Santiago	4440
Guayas	62640
Loja	5974
Galápagos	373
Santa Elena	6547

Export

1 2 3 10 / page

Took 2 seconds. Last updated by anonymous at July 02 2025, 6:47:25 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("cant_res")
    .groupBy("cant_res")
    .count()
}
```

Setting

cant_res	count
Buena Fe	1597
Santiago de Píllaro	485
El Chaco	147
Girón	174
Portovelo	197
Espíndola	170
Tiwintza	272
Limón Indanza	153
Nobol	370
San Cristóbal	105

Export

1 2 3 4 5 ••• 22 10 / page

Took 1 second. Last updated by anonymous at July 02 2025, 6:47:52 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("parr_res")
    .groupBy("parr_res")
    .count()
    .limit(200)
}
```

Setting

parr_res	count
El Recreo	1086
Los Encuentros	67
Chunchi, cabecera cantonal	100
San Vicente de Huaticocha	17
Baeza, cabecera cantonal	29
Alejandro Labaka	15
Bolívar (Sagrario)	257
Carlos Julio Arosemena Tola, cabecera cantonal	82
La Bocana	18
Puerto El Carmen del Putumayo, cabecera cantonal	55

Export

1 2 3 4 5 ••• 20 > 10 / page

Took 1 second. Last updated by anonymous at July 02 2025, 7:22:42 PM.

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("area_res")
    .groupBy("area_res")
    .count()
}
```

Setting

Took 1 second. Last updated by anonymous at July 02 2025, 6:49:26 PM. (outdated)

SPARK JOB FINISHED

```
z.show {
  dfNacidosClean
    .select("residente")
    .groupBy("residente")
    .count()
}
```

Setting

Took 2 seconds. Last updated by anonymous at July 02 2025, 6:50:10 PM. (outdated)

SPARK JOB FINISHED

```
val modaParrRes = dfNacidosClean
  .groupBy("parr_res")
  .count()
  .orderBy(desc("count"))
  .first()
```

```
modaParrRes: org.apache.spark.sql.Row = [Tarqui,17820]
```

Took 1 second. Last updated by anonymous at July 02 2025, 7:16:24 PM.

FINISHED

Semana 15

Took 0 seconds. Last updated by anonymous at July 16 2025, 7:37:38 PM.

SPARK JOB FINISHED

```
val stats = dfNacidosClean
    .select(
        avg("anio_nac").as("media"),
        stddev("anio_nac").as("desviacion_estandar")
    ).first
```

```
stats: org.apache.spark.sql.Row = [2022.9372345054808,1.5180530471163742]
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:16:45 AM.

FINISHED

```
val media = stats.getAs[Double]("media")
val desStand = stats.getAs[Double]("desviacion_estandar")
val umbralDesviacion = 2.0
```

```
media: Double = 2022.9372345054808
desStand: Double = 1.5180530471163742
umbralDesviacion: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:16:48 AM.

FINISHED

```
val limiteInfDS = media - umbralDesviacion * desStand
val limiteSupDS = media + umbralDesviacion * desStand
```

```
limiteInfDS: Double = 2019.901128411248
limiteSupDS: Double = 2025.9733405997135
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:16:51 AM.

FINISHED

```
val outliersDS = dfNacidosClean
    .where($"anio_nac" < limiteInfDS || $"anio_nac" > limiteSupDS)
```

```
outliersDS: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:16:54 AM.

SPARK JOB FINISHED

```
outliersDS.count
```

```
res1: Long = 1028
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:16:59 AM.

SPARK JOB FINISHED

```
val cuartiles = dfNacidosClean.stat.approxQuantile(
    "anio_nac", Array(0.25, 0.75), 0.01
)
```

```
cuartiles: Array[Double] = Array(2023.0, 2023.0)
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:17:03 AM.

FINISHED

```
val q1 = cuartiles(0)
val q3 = cuartiles(1)
val iqr = q3 - q1
```

```
q1: Double = 2023.0
q3: Double = 2023.0
iqr: Double = 0.0
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:17:05 AM.

FINISHED

```
val limiteInferiorIQR = q1 - 1.5 * iqr
val limiteSuperiorIQR = q1 + 1.5 * iqr
```

```
limiteInferiorIQR: Double = 2023.0
limiteSuperiorIQR: Double = 2023.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:17:07 AM.

FINISHED

```
val outliersIQR = dfNacidosClean
    .where($"anio_nac" < limiteInferiorIQR || $"anio_nac" > limiteSuperiorIQR)
```

```
outliersIQR: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:17:11 AM.

SPARK JOB FINISHED

```
outliersIQR.count
```

```
res2: Long = 2523
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:17:14 AM.

FINISHED

```
dfNacidosClean.select("anio_nac")
    .createOrReplaceTempView("dfNacidosAnioNac")
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 11:06:32 AM.

FINISHED

```
%pyspark

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:17:23 AM.

SPARK JOB FINISHED

```
%pyspark
tmp_view_nacidos = spark.sql("SELECT * FROM dfNacidosAnioNac")
df_nacidos_anio_nac = tmp_view_nacidos.toPandas()
```

Took 3 seconds. Last updated by anonymous at July 17 2025, 10:17:29 AM.

FINISHED

```
%pyspark
z.show(df_nacidos_anio_nac)
```

Setting

anio_nac
2005
2006
2007
2007
2008
2009
2010
2010
2011
2008

Export

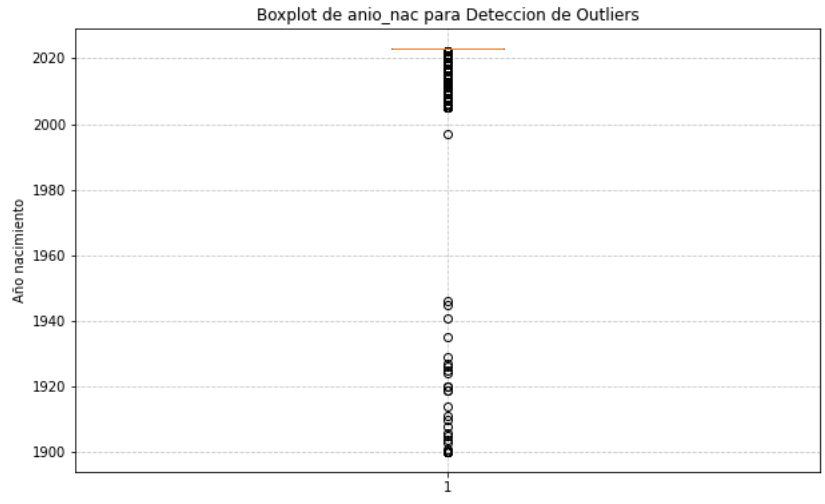
1 2 3 4 5 ... 100 > 10 / page

Results are limited by 1000.

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:17:30 AM.

FINISHED

```
%pyspark

plt.figure(figsize = (10, 6))
plt.boxplot(df_nacidos_anio_nac['anio_nac'],showfliers=True)
plt.title('Boxplot de anio_nac para Deteccion de Outliers')
plt.ylabel('Año nacimiento')
plt.grid(True, linestyle='--', alpha = 0.7)
plt.show()
```



Took 0 seconds. Last updated by anonymous at July 17 2025, 10:17:37 AM.

FINISHED

Semana 15-Parte2

Took 0 seconds. Last updated by anonymous at July 16 2025, 7:41:16 PM.

SPARK JOB FINISHED

```
val stats = dfNacidosClean
    .select(
        avg("anio_nac").as("media"),
        stddev("anio_nac").as("desviacion_estandar")
    ).first
```

```
stats: org.apache.spark.sql.Row = [2022.9372345054808,1.5180530471163742]
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:54:32 AM.

FINISHED

```
val media = stats.getAs[Double]("media")
val desStand = stats.getAs[Double]("desviacion_estandar")
val umbralDesviacion = 2.0
```

```
media: Double = 2022.9372345054808
desStand: Double = 1.5180530471163742
umbralDesviacion: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:33 AM.

FINISHED

```
val limiteInfDS = media - umbralDesviacion * desStand
val limiteSupDS = media + umbralDesviacion * desStand
```

```
limiteInfDS: Double = 2019.901128411248
limiteSupDS: Double = 2025.9733405997135
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:35 AM.

FINISHED

```
val outliersDS = dfNacidosClean
    .where($"dia_nac" < limiteInfDS || $"dia_nac" > limiteSupDS)
```

```
outliersDS: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:36 AM.

SPARK JOB FINISHED

```
outliersDS.count
```

```
res17: Long = 241295
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:54:39 AM.

SPARK JOB FINISHED

```
val cuartiles = dfNacidosClean.stat.approxQuantile(
    "dia_nac", Array(0.25, 0.75), 0.01
)
```

```
cuartiles: Array[Double] = Array(8.0, 23.0)
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:54:41 AM.

FINISHED

```
val q1 = cuartiles(0)
val q3 = cuartiles(1)
val iqr = q3 - q1
```

```
q1: Double = 8.0
q3: Double = 23.0
```

```
iqr: Double = 15.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:42 AM.

FINISHED

```
val limiteInferiorIQR = q1 - 1.5 * iqr
val limiteSuperiorIQR = q1 + 1.5 * iqr
```

```
limiteInferiorIQR: Double = -14.5
limiteSuperiorIQR: Double = 30.5
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:46 AM.

FINISHED

```
val outliersIQR = dfNacidosClean
    .where($"dia_nac" < limiteInferiorIQR || $"dia_nac" > limiteSuperiorIQR)
```

```
outliersIQR: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:48 AM.

SPARK JOB FINISHED

```
outliersIQR.count
```

```
res18: Long = 4556
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:54:52 AM.

FINISHED

```
dfNacidosClean.select("dia_nac")
    .createOrReplaceTempView("dfNacidosAnioNac")
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:54:53 AM.

SPARK JOB FINISHED

```
%pyspark
tmp_view_nacidos = spark.sql("SELECT * FROM dfNacidosAnioNac WHERE dia_nac is not null")
df_nacidos_anio_nac = tmp_view_nacidos.toPandas()
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:55:07 AM.

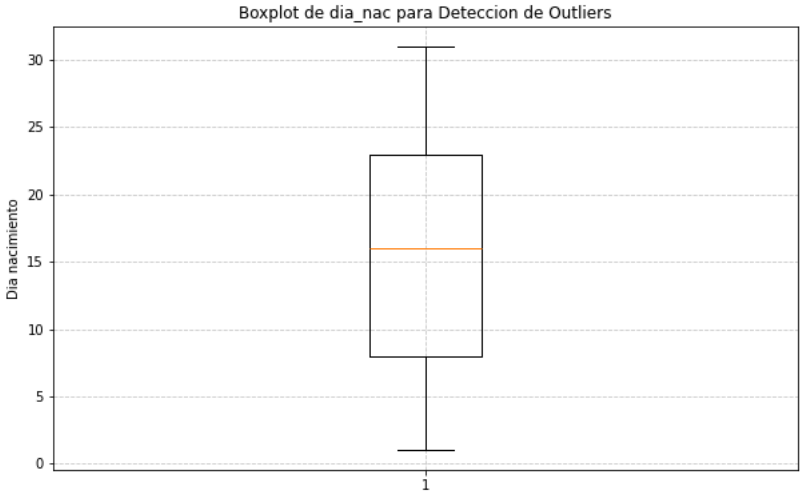
FINISHED

```
%pyspark

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

Took 1 second. Last updated by anonymous at July 16 2025, 8:44:58 PM.

FINISHED

```
%pyspark

plt.figure(figsize = (10, 6))
plt.boxplot(df_nacidos_anio_nac['dia_nac'],showfliers=True)
plt.title('Boxplot de dia_nac para Deteccion de Outliers')
plt.ylabel('Dia nacimiento')
plt.grid(True, linestyle='--', alpha = 0.7)
plt.show()
```



Took 0 seconds. Last updated by anonymous at July 17 2025, 10:55:22 AM.

SPARK JOB FINISHED

```
val stats = dfNacidosClean
    .select(
        avg("hij_viv").as("media"),
        stddev("hij_viv").as("desviacion_estandar")
    ).first
```

```
stats: org.apache.spark.sql.Row = [2.0577550301498166,1.2242374055754184]
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:55:59 AM.

FINISHED

```
val media = stats.getAs[Double]("media")
val desStand = stats.getAs[Double]("desviacion_estandar")
val umbralDesviacion = 2.0
```

```
media: Double = 2.0577550301498166
desStand: Double = 1.2242374055754184
umbralDesviacion: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:00 AM.

FINISHED

```
val limiteInfDS = media - umbralDesviacion * desStand
val limiteSupDS = media + umbralDesviacion * desStand
```

```
limiteInfDS: Double = -0.3907197810010201
limiteSupDS: Double = 4.506229841300653
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:02 AM.

FINISHED

```
val outliersDS = dfNacidosClean
    .where($"hij_viv" < limiteInfDS || $"hij_viv" > limiteSupDS)
```

```
outliersDS: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:04 AM.

SPARK JOB FINISHED

```
outliersDS.count
```

```
res20: Long = 10032
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:07 AM.

SPARK JOB FINISHED

```
val cuartiles = dfNacidosClean.stat.approxQuantile(
    "hij_viv", Array(0.25, 0.75), 0.01
)
```

```
cuartiles: Array[Double] = Array(1.0, 3.0)
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:09 AM.

FINISHED

```
val q1 = cuartiles(0)
val q3 = cuartiles(1)
val iqr = q3 - q1
```

```
q1: Double = 1.0
q3: Double = 3.0
iqr: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:09 AM.

FINISHED

```
val limiteInferiorIQR = q1 - 1.5 * iqr
val limiteSuperiorIQR = q1 + 1.5 * iqr
```

```
limiteInferiorIQR: Double = -2.0
limiteSuperiorIQR: Double = 4.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:11 AM.

FINISHED

```
val outliersIQR = dfNacidosClean
    .where($"hij_viv" < limiteInferiorIQR || $"hij_viv" > limiteSuperiorIQR)
```

```
outliersIQR: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:13 AM.

SPARK JOB FINISHED

```
outliersIQR.count
```

```
res21: Long = 10032
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:16 AM.

FINISHED

```
dfNacidosClean.select("hij_viv")
    .createOrReplaceTempView("dfNacidosAnioNac")
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:16 AM.

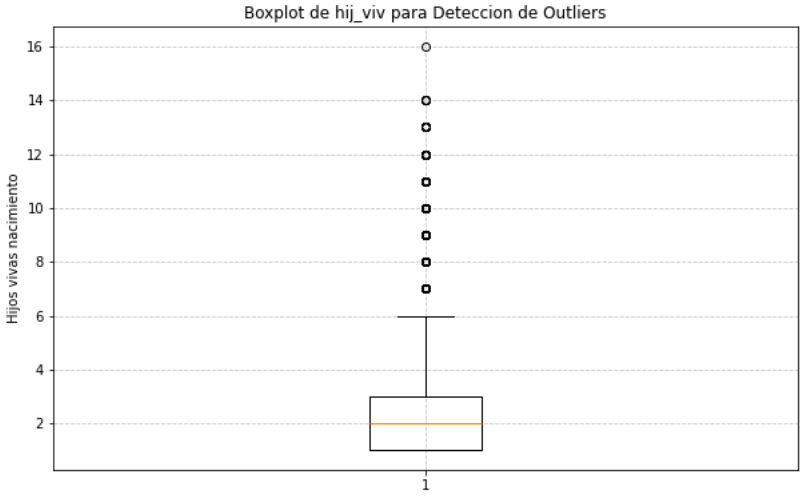
SPARK JOB FINISHED

```
%pyspark
tmp_view_nacidos = spark.sql("SELECT * FROM dfNacidosAnioNac")
df_nacidos_anio_nac = tmp_view_nacidos.toPandas()
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:56:25 AM.

FINISHED

```
%pyspark

plt.figure(figsize = (10, 6))
plt.boxplot(df_nacidos_anio_nac['hij_viv'],showfliers=True)
plt.title('Boxplot de hij_viv para Deteccion de Outliers')
plt.ylabel('Hijos vivas nacimiento')
plt.grid(True, linestyle='--', alpha = 0.7)
plt.show()
```



Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:27 AM.

FINISHED

```
dfNacidosCleanFinal.printSchema
```

```
root
 |-- prov_insc: string (nullable = true)
 |-- cant_insc: string (nullable = true)
 |-- parr_insc: string (nullable = true)
 |-- fecha_insc: string (nullable = true)
 |-- anio_insc: integer (nullable = true)
 |-- mes_insc: string (nullable = true)
 |-- dia_insc: integer (nullable = true)
 |-- sexo: string (nullable = true)
 |-- fecha_nac: string (nullable = true)
 |-- anio_nac: integer (nullable = true)
 |-- mes_nac: string (nullable = true)
 |-- dia_nac: integer (nullable = true)
 |-- talla: integer (nullable = true)
 |-- peso: integer (nullable = true)
 |-- sem_gest: integer (nullable = true)
 |-- tipo_part: string (nullable = true)
 |-- lugar_ocur: string (nullable = true)
 |-- apgar1: integer (nullable = true)
 |-- apgar5: integer (nullable = true)
 |-- p_emb: string (nullable = true)
 |-- prov_nac: string (nullable = true)
 |-- cant_nac: string (nullable = true)
 |-- parr_nac: string (nullable = true)
 |-- area_nac: string (nullable = true)
 |-- asis_por: string (nullable = true)
 |-- nac_mad: string (nullable = true)
 |-- cod_pais: string (nullable = true)
 |-- fecha_mad: string (nullable = true)
 |-- anio_mad: integer (nullable = true)
 |-- mes_mad: string (nullable = true)
 |-- dia_mad: integer (nullable = true)
 |-- edad_mad: integer (nullable = true)
 |-- con_pren: integer (nullable = true)
 |-- num_emb: integer (nullable = true)
 |-- num_par: integer (nullable = true)
 |-- hij_viv: integer (nullable = true)
 |-- hij_vivm: integer (nullable = true)
 |-- hij_nacm: integer (nullable = true)
 |-- etnia: string (nullable = true)
 |-- est_civil: string (nullable = true)
 |-- niv_inst: string (nullable = true)
 |-- sabe_leer: string (nullable = true)
 |-- prov_res: string (nullable = true)
 |-- cant_res: string (nullable = true)
 |-- parr_res: string (nullable = true)
 |-- area_res: string (nullable = true)
 |-- residente: string (nullable = true)
 |-- fecha_insc_date: date (nullable = true)
 |-- fecha_nac_date: date (nullable = true)
 |-- fecha_mad_date: date (nullable = true)
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:21:03 AM.

SPARK JOB FINISHED

```
val stats = dfNacidosCleanFinal
    .select(
        avg("apgar1").as("media"),
        stddev("apgar1").as("desviacion_estandar")
    ).first
```

```
stats: org.apache.spark.sql.Row = [8.06699478915039,0.8670639431455726]
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:42 AM.

FINISHED

```
val media = stats.getAs[Double]("media")
val desStand = stats.getAs[Double]("desviacion_estandar")
val umbralDesviacion = 2.0
```

```
media: Double = 8.06699478915039
desStand: Double = 0.8670639431455726
umbralDesviacion: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:43 AM.

FINISHED

```
val limiteInfDS = media - umbralDesviacion * desStand
val limiteSupDS = media + umbralDesviacion * desStand
```

```
limiteInfDS: Double = 6.332866902859245
limiteSupDS: Double = 9.801122675441535
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:45 AM.

FINISHED

```
val outliersDS = dfNacidosCleanFinal
    .where($"apgar1" < limiteInfDS || $"apgar1" > limiteSupDS)
```

```
outliersDS: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:46 AM.

SPARK JOB FINISHED

```
outliersDS.count
```

```
res23: Long = 9136
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:51 AM.

SPARK JOB FINISHED

```
val cuartiles = dfNacidosCleanFinal.stat.approxQuantile(
    "apgar1", Array(0.25, 0.75), 0.01
)
```

```
cuartiles: Array[Double] = Array(8.0, 9.0)
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:56:57 AM.

FINISHED

```
val q1 = cuartiles(0)
val q3 = cuartiles(1)
val iqr = q3 - q1
```

```
q1: Double = 8.0
q3: Double = 9.0
iqr: Double = 1.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:56:58 AM.

FINISHED

```
val limiteInferiorIQR = q1 - 1.5 * iqr
val limiteSuperiorIQR = q1 + 1.5 * iqr
```

```
limiteInferiorIQR: Double = 6.5
limiteSuperiorIQR: Double = 9.5
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:57:00 AM.

FINISHED

```
val outliersIQR = dfNacidosCleanFinal
    .where($"apgar1" < limiteInferiorIQR || $"apgar1" > limiteSuperiorIQR)
```

```
outliersIQR: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:57:02 AM.

SPARK JOB FINISHED

```
outliersIQR.count
```

```
res24: Long = 9136
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:57:04 AM.

FINISHED

```
dfNacidosCleanFinal.select("apgar1")
    .createOrReplaceTempView("dfNacidosAnioNac")
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:57:05 AM.

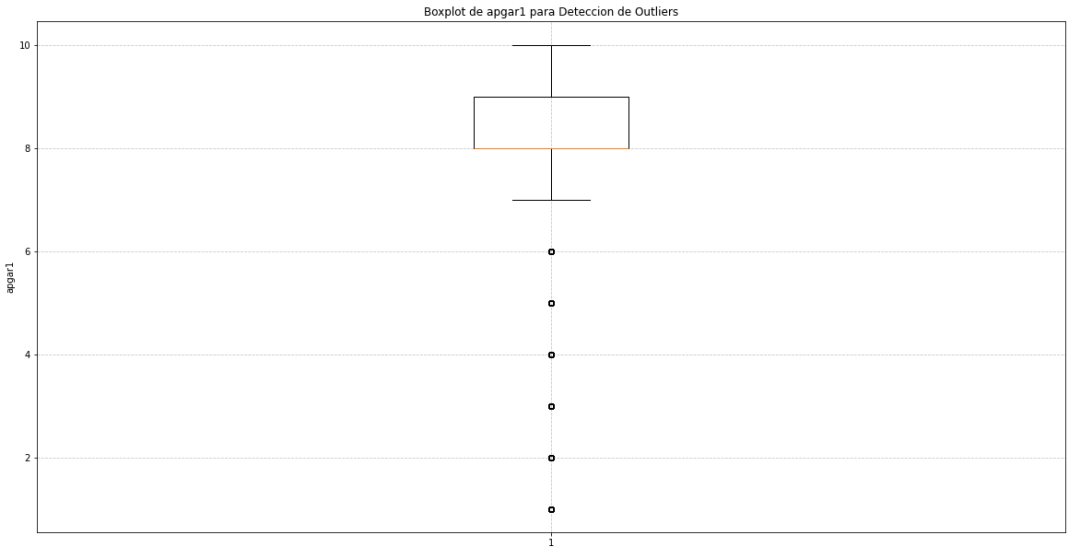
SPARK JOB FINISHED

```
%pyspark

tmp_view_nacidos = spark.sql("SELECT * FROM dfNacidosAnioNac WHERE apgar1 IS NOT NULL")
df_nacidos_anio_nac = tmp_view_nacidos.toPandas()
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:57:09 AM.

FINISHED

```
%pyspark

plt.figure(figsize = (20, 10))
plt.boxplot(df_nacidos_anio_nac['apgar1'],showfliers=True)
plt.title('Boxplot de apgar1 para Deteccion de Outliers')
plt.ylabel('apgar1')
plt.grid(True, linestyle='--', alpha = 0.7)
plt.show()
```



Took 0 seconds. Last updated by anonymous at July 17 2025, 10:57:21 AM.

SPARK JOB FINISHED

```
val stats = dfNacidosCleanFinal
    .select(
        avg("peso").as("media"),
        stddev("peso").as("desviacion_estandar")
    ).first
```

```
stats: org.apache.spark.sql.Row = [3068.7153236346126,505.60786527826025]
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:58:23 AM.

FINISHED

```
val media = stats.getAs[Double]("media")
val desStand = stats.getAs[Double]("desviacion_estandar")
val umbralDesviacion = 2.0
```

```
media: Double = 3068.7153236346126
desStand: Double = 505.60786527826025
umbralDesviacion: Double = 2.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:58:31 AM.

FINISHED

```
val limiteInfDS = media - umbralDesviacion * desStand
val limiteSupDS = media + umbralDesviacion * desStand
```

```
limiteInfDS: Double = 2057.499593078092
limiteSupDS: Double = 4079.931054191133
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:58:38 AM.

FINISHED

```
val outliersDS = dfNacidosCleanFinal
    .where($"peso" < limiteInfDS || $"peso" > limiteSupDS)
```

```
outliersDS: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:59:01 AM.

SPARK JOB FINISHED

```
outliersDS.count
```

```
res26: Long = 11647
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:59:11 AM.

SPARK JOB FINISHED

```
val cuartiles = dfNacidosCleanFinal.stat.approxQuantile(
    "peso", Array(0.25, 0.75), 0.01

)
```

```
cuartiles: Array[Double] = Array(2800.0, 3385.0)
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 10:59:25 AM.

FINISHED

```
val q1 = cuartiles(0)
val q3 = cuartiles(1)
val iqr = q3 - q1
```

```
q1: Double = 2800.0
q3: Double = 3385.0
iqr: Double = 585.0
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 10:59:34 AM.

FINISHED

```
val limiteInferiorIQR = q1 - 1.5 * iqr
val limiteSuperiorIQR = q1 + 1.5 * iqr
```

```
limiteInferiorIQR: Double = 1922.5
limiteSuperiorIQR: Double = 3677.5
```

Took 1 second. Last updated by anonymous at July 17 2025, 10:59:45 AM.

FINISHED

```
val outliersIQR = dfNacidosCleanFinal
    .where($"peso" < limiteInferiorIQR || $"peso" > limiteSuperiorIQR)
```

```
outliersIQR: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [prov_insc: string, cant_insc: string ... 48 more fields]
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 11:00:03 AM.

SPARK JOB FINISHED

```
outliersIQR.count
```

```
res27: Long = 27529
```

Took 2 seconds. Last updated by anonymous at July 17 2025, 11:00:11 AM.

FINISHED

```
dfNacidosCleanFinal.select("peso")
    .createOrReplaceTempView("dfNacidosAnioNac")
```

Took 0 seconds. Last updated by anonymous at July 17 2025, 11:00:21 AM.

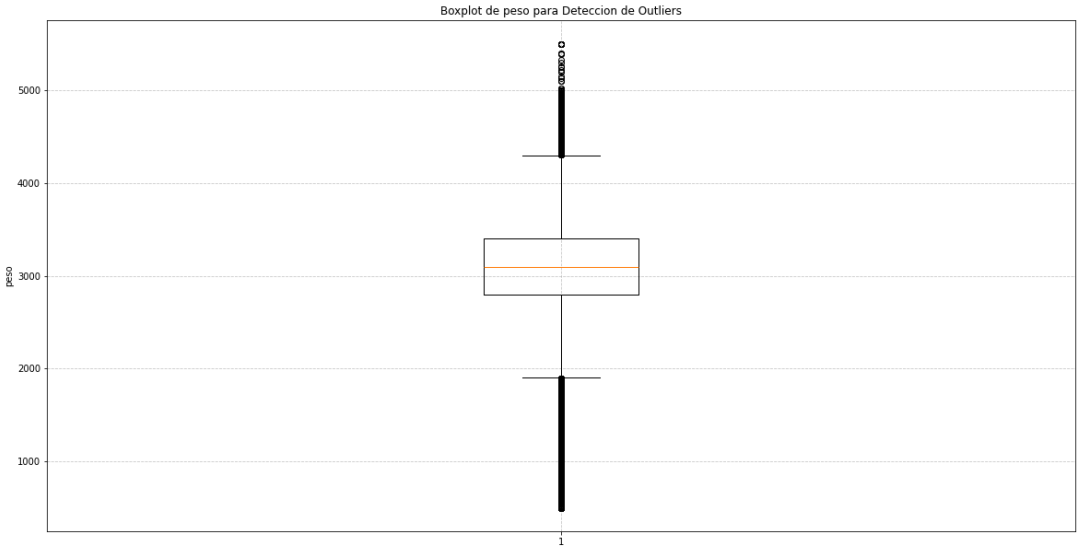
SPARK JOB FINISHED

```
%pyspark

tmp_view_nacidos = spark.sql("SELECT * FROM dfNacidosAnioNac WHERE peso IS NOT NULL")
df_nacidos_anio_nac = tmp_view_nacidos.toPandas()
```

Took 4 seconds. Last updated by anonymous at July 17 2025, 11:00:50 AM.

FINISHED

```
%pyspark

plt.figure(figsize = (20, 10))
plt.boxplot(df_nacidos_anio_nac['peso'],showfliers=True)
plt.title('Boxplot de peso para Deteccion de Outliers')
plt.ylabel('peso')
plt.grid(True, linestyle='--', alpha = 0.7)
plt.show()
```



Took 0 seconds. Last updated by anonymous at July 17 2025, 11:06:53 AM.

READY

```
%pyspark
```